FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de tratamiento de Datos

SQL Avanzado

Solución Ejercicios

Año: 2025

FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

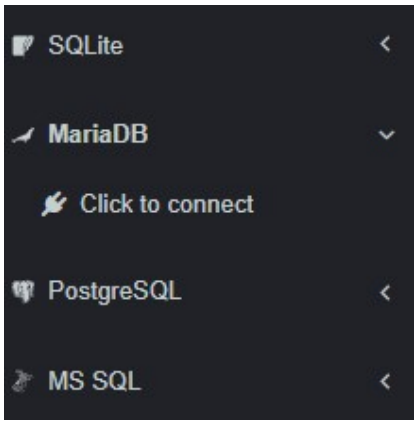
Soluciones

Para practicar:

Para aquellos que les interese probar interactivamente la ejecución de las sentencias, se sugiere el siguiente sitio: **sqliteonline.com**

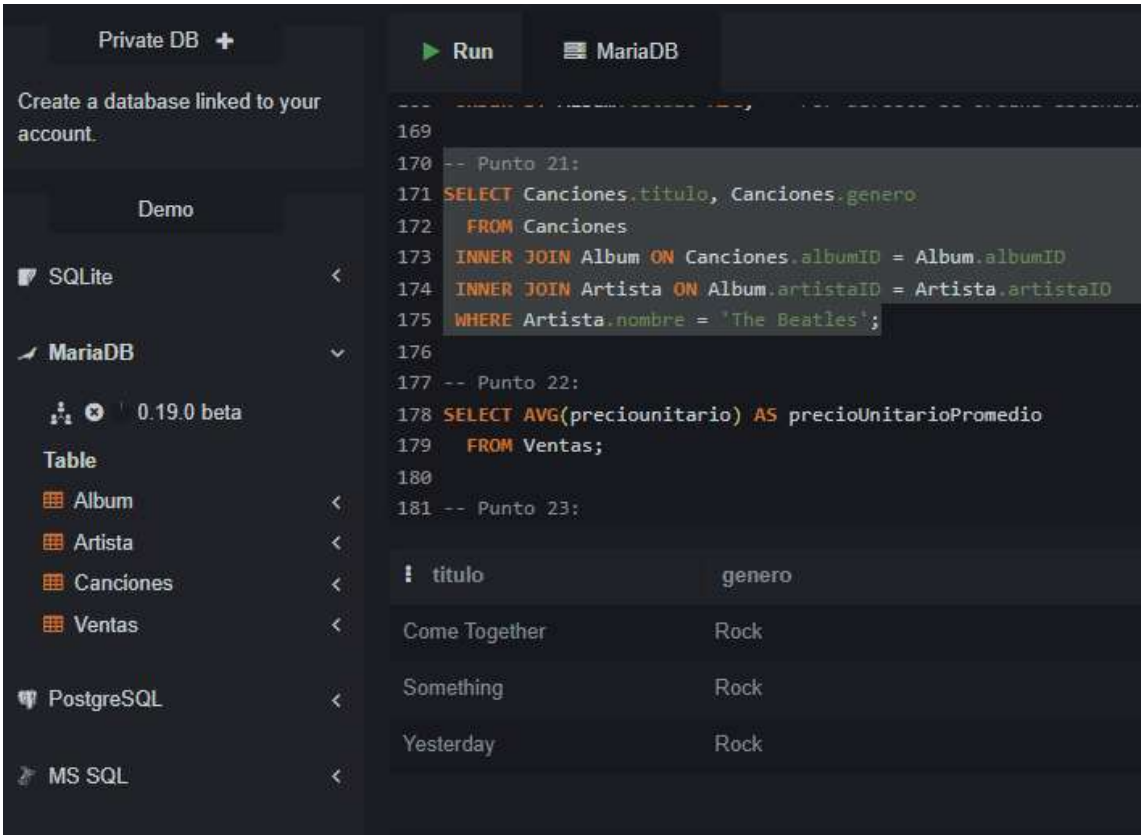
Es una herramienta en línea (editor/compilador) que permite a los usuarios interactuar con bases de datos directamente desde un navegador web, sin necesidad de instalar el motor de base de datos en su computadora.

Se pueden usar diferentes alternativas de DBMS: SQLite, MariaDB, PostgreSQL y MS SQL (SQL Server). Simplemente hay que tocar en "Click to connect" en el desplegable de cada alternativa. Por ejemplo, para usar MariaDB:



En todos los casos usará una base de datos por defecto que incluye una tabla “demo” (puede eliminarse, pero a fines prácticos no hace falta).

Sectores básicos para usar:



- En el menú de la izquierda, se presentan las alternativas de DMBS. Para cada una, las tablas que se vayan creando en su base de datos.
- En la ventana de la derecha, en la parte superior se pueden ir escribiendo las sentencias SQL. Para ejecutarlas, hay que seleccionar la o las sentencias que interesen, y dar click en Run (flecha verde).

- En la ventana de la derecha, en la parte inferior, se mostrarán los resultados de las sentencias ejecutadas.

Se presentan a continuación las soluciones a los ejercicios propuestos. Los textos que no corresponden con sentencias se incluyeron con "--" antes, para que no sean interpretados como comandos (permite agregar comentarios en los scripts de SQL).

De esta manera, **se pueden copiar y pegar las soluciones propuestas en la consola para ir probando interactivamente las sentencias** (seleccionando de a una o varias y ejecutando).

Además, en la plataforma encontrarán el archivo **"SQL Avanzado Soluciones Ejercicios.sql",** que tiene las sentencias para seguir las soluciones de los ejercicios propuestos en la práctica. Pueden abrirlo en un editor de texto y pegar su contenido en la ventana para su ejecución, o bien abrirlo desde el menú.

Aquellos que no tengan la posibilidad de utilizar la herramienta propuesta, o cualquier otro entorno de base de datos con el que dispongan (o prefieran), podrán ir leyendo de todas formas para seguir el razonamiento.

```
-- ---------------------------------- --
-- Subconsultas en la cláusula WHERE --
-- ---------------------------------- --

-- Pregunta 1 (Uso del operador "="): Encontrar el título del álbum con la
-- mayor cantidad de unidades en una única venta.

SELECT AL.Titulo
  FROM Albumes AL
 INNER JOIN Ventas V ON (AL.AlbumID = V.AlbumID)
WHERE V.Cantidad = ( SELECT MAX(Cantidad)
                       FROM Ventas
                   );

-- La subconsulta devuelve una única fila y columna con el máximo de unidades
-- venidas en una venta.
-- Este resultado es usado por la consulta externa con el operador "=".
-- El INNER JOIN se necesita para poder mostrar los títulos del/los álbumes.

-- Pregunta 2 (Uso del operador ">"): Encontrar el título, el género y el
-- nombre del álbum de las canciones que duren más que el promedio de las
-- canciones de género "POP".

SELECT Canciones.Titulo, Genero, Albumes.Titulo, Duracion_segundos
  FROM Canciones
 INNER JOIN Albumes ON (Canciones.albumID = Albumes.AlbumID)
WHERE Duracion_segundos > ( SELECT AVG(Duracion_segundos)
                              FROM Canciones
                             WHERE Genero = 'POP'
                          );
```

```
-- Ver que el INNER JOIN se hace sólo para obtener el nombre del álbum al que
-- pertenece la canción.
-- Se agrega el prefijo en las columnas a mostrar porque tienen el mismo nombre.
-- La subconsulta devuelve una única fila y una única columna con la duración
-- promedio de las canciones de 'POP'.
-- Este resultado es usado por la consulta principal con el operador ">".
-- Se puede verificar ejecutando la subconsulta para ver su resultado,
-- y agregar el valor de duración en segundos para ver que se cumple.

-- Pregunta 3 (Uso del operador "IN"): Encontrar el título y año de lanzamiento
-- de los álbumes cuyos artistas sean de Reino Unido o Estados Unidos.

SELECT Titulo, AnioLanzamiento
  FROM Albumes
WHERE ArtistaID IN ( SELECT ArtistaID
                       FROM Artistas
                      WHERE Pais IN ('Reino Unido', 'Estados Unidos')
                   );
-- La subconsulta devuelve una columna y varias filas, con los ID de los
-- artistas de los países indicados.
-- El resultado es usado por la consulta principal con el operador "IN",
-- para verificar si está en esa lista.
-- Se podría mostrar un ejemplo de una expresión con dos valores
-- (artistaID, año) y compara contra año nacimiento.

-- Pregunta 4 (Uso del operador "NOT IN"): Encontrar los artistas que
-- no han ganado premios.
```

```
SELECT *
  FROM Artistas
WHERE ArtistaID NOT IN ( SELECT ArtistaID
                           FROM Premios
                       );

-- La subconsulta devuelve una columna y varias filas, con los ID de los
-- artistas que ganaron premios.
-- El resultado es usado por la consulta externa con el operador "NOT IN",
-- para verificar que no esté en esa lista.

-- Pregunta 5 (Uso del operador "ANY"): Obtener los títulos de las canciones
-- con una duración menor que cualquiera (ALGUNA) de las canciones del álbum
-- 'A Night at The Opera'.

SELECT Titulo
  FROM Canciones
WHERE Duracion_segundos < ANY ( SELECT Duracion_segundos
                                  FROM Canciones C
                                 INNER JOIN Albumes AL
                                    ON (C.AlbumID = AL.AlbumID)
                                 WHERE AL.Titulo = 'A Night at The Opera'
                               );

-- La subconsulta devuelve las duraciones de 'Bohemian Rhapsody' (355s)
-- y 'Don"t Stop Me Now' (209s).
-- La consulta principal busca cualquier canción cuya duración sea menor
-- que 355 o 209, es decir, cualquier canción con menos de 355 segundos.
-- La mayoría de las canciones cumplen esta condición.
```

```
-- Pregunta 6 (Uso del operador "ALL"): Mostrar título de álbum, fecha de venta,
-- y monto total (Cantidad * Precio); donde el monto total es mayor o igual
-- a los montos totales de TODAS las ventas realizadas a partir del 1 de junio de 2025.

SELECT Albumes.titulo, FechaVenta, (Cantidad * Precio) AS monto_total
  FROM Albumes
 INNER JOIN Ventas ON (Albumes.AlbumID = Ventas.AlbumID)
WHERE (Cantidad * Precio) >= ALL ( SELECT (Cantidad * Precio)
                                     FROM Ventas
                                    WHERE FechaVenta >= '2025-06-01'
                                  );

-- La subconsulta devuelve la lista del precio total de las ventas
-- realizadas a partir del 1 de junio de 2025.
-- La consulta externa usa este resultado para verificar cuales álbumes
-- tuvieron un precio mayor o igual a TODOS los de la subconsulta.

-- Pregunta 7 (Uso del operador "EXISTS"): Encontrar el título de las canciones
-- y el nombre del álbum, de las canciones que estén en álbumes para los que en
-- una misma venta se hayan vendido más de 5 unidades.

SELECT C.Titulo, AL.Titulo
  FROM Canciones C
 INNER JOIN Albumes AL ON (C.AlbumID = AL.AlbumID)
WHERE EXISTS ( SELECT *
                 FROM Ventas V
                WHERE V.AlbumID = AL.AlbumID
                  AND V.Cantidad > 5
             );
```

```
-- La subconsulta en este caso es correlacionada (V.AlbumID = AL.AlbumID),
-- y se ejecuta para cada registro de la consulta principal.
-- En el caso del operador booleano EXIST, lo que importa es si la subconsulta
-- devuelve o no algún registro, por lo que en lugar de SELECT * se podría haber
-- usado SELECT 1 o cualquier otro campo.
-- En cuanto la subconsulta encuentre un registro en Ventas con cantidad > 5 para
-- el Álbum al que hace referencia, el predicado será verdadero (TRUE).

-- Pregunta 8 (Uso del operador "NOT EXISTS"): Encontrar los álbumes cuyo año
-- de lanzamiento no sea el año de nacimiento de un artista.

SELECT *
  FROM Albumes AL
WHERE NOT EXISTS ( SELECT 1
                     FROM Artistas AR
                    WHERE AL.AnioLanzamiento = YEAR(AR.FechaNacimiento)
                  );

-- La subconsulta en este caso es correlacionada porque hace uso de un valor de la
-- consulta principal (AL.AnioLanzamiento = YEAR(AR.FechaNacimiento), y se ejecuta
-- para cada registro de la consulta principal.
-- En cuanto la subconsulta encuentre un registro donde el año de lanzamiento
-- coincida con el año de nacimiento de un artista, el predicado será falso (FALSE).
-- Ver que en este caso se usa SELECT 1, porque lo único que interesa
-- es si devuelve registros o no.

-- EXTRA: ¿Qué pasaría si quisiéramos hacer lo mismo con el operador “NOT IN”?
-- El operador NOT IN tiene un comportamiento particular si la subconsulta
-- devuelve algún nulo.
-- Si ocurre esto, el resultado del predicado será "desconocido", y la consulta
```

```
-- principal no devolverá registros.
-- En este caso, existen Artistas (los grupos) con fecha de nacimiento nula.
-- Para evitar ese comportamiento, deberíamos asegurarnos que la subconsulta
-- no devuelva nulos, bien con el uso de la función COALESCE, o con una
-- cláusula WHERE que filtre los registros para que no aparezcan los que
-- tienen fecha de nacimiento NULL.

SELECT *
  FROM Albumes
WHERE AnioLanzamiento NOT IN
                              ( SELECT COALESCE(YEAR(FechaNacimiento), '1000')
                                  FROM Artistas
                                -- WHERE FechaNacimiento IS NOT NULL
                               );

-- Si no incluyéramos COALESCE o WHERE ... IS NOT NULL en la subconsulta,
-- la principal no traería resultados.

-- ---------------------------------- --
-- Subconsultas en la cláusula SELECT --
-- ---------------------------------- --

-- Pregunta 9: Seleccionar título, género, duración de las canciones, y el promedio
-- de la duración de las canciones de su mismo género.

SELECT Titulo, Genero, Duracion_Segundos
          , ( SELECT AVG(Duracion_Segundos)
                FROM Canciones AS C2
              WHERE C1.Genero = C2.Genero) AS duracion_promedio_genero
  FROM Canciones as C1
ORDER BY Genero;
```

```
-- La subconsulta en este caso es correlacionada (C1.Genero = C2.Genero),
-- y devuelve el promedio de la duración de las canciones que son del mismo género
-- de cada canción de la consulta principal.
-- Ver que se usaron alias para poder usar la misma tabla más de una vez.
-- Tener en cuenta que para usar una subconsulta en la sentencia SELECT,
-- debe devolver un valor (subconsulta escalar: una fila y una columna).
-- Se ordenó el resultado por género para comprobar que el valor agrupado
-- se mantiene para las canciones del mismo género.

-- Pregunta 10: A la anterior, agregar la duración máxima y mínima de las canciones
-- de su mismo género.
-- Para esto, se agregan dos subconsultas en la sentencia SELECT similares a la
-- creada para el AVG, para cada valor que se quiere mostrar (MAX y MIN).

SELECT Titulo, Genero, Duracion_Segundos
        , ( SELECT AVG(Duracion_Segundos)
              FROM Canciones AS C2
             WHERE C1.Genero = C2.Genero) AS duracion_promedio_genero
        , ( SELECT MAX(Duracion_Segundos)
              FROM Canciones AS C2
             WHERE C1.Genero = C2.Genero) AS duracion_maxima_genero
        , ( SELECT MIN(Duracion_Segundos)
              FROM Canciones AS C2
             WHERE C1.Genero = C2.Genero) AS duracion_minima_genero
  FROM Canciones as C1
 ORDER BY Genero;
```

```
-- ------------------------------ --
-- Subconsultas en la cláusula FROM --
-- ------------------------------ --

-- Pregunta 11: Mostrar el nombre del artista y el monto promedio de ingresos
-- totales obtenidos por cada álbum de cada artista.

-- Lógica a seguir:
-- Paso 1 (Subconsulta): Calcular el total de ingresos por cada álbum,
-- agrupando las ventas por AlbumID y sumando el producto de Cantidad * Precio.
-- Esto crea una "tabla derivada" que se usa en la consulta externa.
-- Paso 2 (Consulta Principal): Unir (JOIN) la tabla Albumes con la "tala derivada"
-- a la que se le da un alias (VentasPorAlbum). Además, debe haber un JOIN entre
-- la tabla Artista y la tabla Albumes.
-- Paso 3 (Resultado): Agrupar los resultados por Artista.Nombre y calcular
-- el promedio de los ingresos totales de sus álbumes (obtenidos en la subconsulta).

SELECT A.Nombre AS Artista
          , AVG(VentasPorAlbum.IngresosTotales) AS PromedioVentasAlbum
 FROM Artistas AS A
INNER JOIN Albumes AS AL ON (A.ArtistaID = AL.ArtistaID)
INNER JOIN ( SELECT AlbumID, SUM(Cantidad * Precio) AS IngresosTotales
                    FROM Ventas
                   GROUP BY AlbumID) AS VentasPorAlbum
            ON (AL.AlbumID = VentasPorAlbum.AlbumID)
GROUP BY A.Nombre
ORDER BY PromedioVentasAlbum DESC;

-- La subconsulta dentro del FROM es lo que se conoce como una "tabla derivada".
-- Se le da el alias "VentasPorAlbum" para referenciarla en la consulta principal.
```

```
-- Con "GROUP BY AlbumID" agrupa las ventas de cada álbum, y calcula el total
-- de ingresos de estos con "SUM(Cantidad * Precio)".
-- La tabla derivada se une a la tabla de Albumes con
-- "JOIN ... ON (AL.AlbumID = VentasPorAlbum.AlbumID)". Esto permite vincular los
-- ingresos de los áblumes con la información de los artistas (a través de otro JOIN
-- entre Albumes y Artistas).
-- Finalmente, la consulta externa agurpa por los nombres de artistas, y
-- obtiene el promedio de los ingresos totales obtenidos en la subconsulta
-- con "AVG(VentasPorAlbum.IngresosTotales)".

-- ------------------------------------------------ --
-- Operadores de conjunto UNION, INTERSECT, EXCEPT --
-- ------------------------------------------------ --

-- Pregunta 12: Encontrar los álbumes lanzados antes del 2000
-- más los lanzados después del 2020:

SELECT * from Albumes where AnioLanzamiento < 2000
 UNION
SELECT * FROM Albumes where AnioLanzamiento > 2020
 ORDER BY AnioLanzamiento;

-- Pregunta 13: Encontrar el nombre y país de las personas que son artistas
-- y también fans destacados:

SELECT Nombre, Pais FROM Artistas
INTERSECT
SELECT Nombre, Pais FROM FansDestacados;

-- Ver que, aunque son campos de tablas diferentes, ambas sentencias SELECT
-- devuelven la misma cantidad de columnas (2) y estas son del mismo tipo de datos.
```

-- **Pregunta 14:** Mostrar el nombre y país de las personas que son artistas
-- pero no son fans destacados:

```
SELECT Nombre, Pais from Artistas
EXCEPT
SELECT Nombre, Pais FROM FansDestacados;
```

-- **Pregunta 15:** Encontrar personas que son fans destacados
-- pero no son artistas:

```
SELECT Nombre, Pais FROM FansDestacados
EXCEPT
SELECT Nombre, Pais from Artistas;
```

-- Se observa en estas dos últimas preguntas que el operador EXCEPT no es conmutativo.

```
-- ----------------------- --
-- Creación y uso de vistas --
-- ----------------------- --

-- Pregunta 16: Crear una vista llamada ResumenAlbumes que combine información
-- de las tablas Artistas, Albumes, Canciones y Ventas para mostrar un resumen
-- de cada álbum, incluyendo el nombre del artista, el título del álbum, el año
-- de lanzamiento, el número de canciones, y el total de ingresos por ventas.

CREATE VIEW ResumenAlbumes AS
SELECT A.Nombre AS NombreArtista,
       AL.Titulo AS TituloAlbum,
       AL.AnioLanzamiento,
       COUNT(C.CancionID) AS NumeroCanciones,
       SUM(V.Cantidad * V.Precio) AS IngresosTotales
  FROM Artistas AS A
 INNER JOIN Albumes AS AL ON A.ArtistaID = AL.ArtistaID
  LEFT JOIN Canciones AS C ON AL.AlbumID = C.AlbumID
  LEFT JOIN Ventas AS V ON AL.AlbumID = V.AlbumID
GROUP BY A.Nombre, AL.Titulo, AL.AnioLanzamiento;

-- Con "CREATE VIEW ResumenAlbumes AS" se crea la vista y se le da un nombre.
-- En este caso, toma los nombres de columna directamente del SELECT, pero podría
-- haberse asignado otros nombres entre paréntesis a la derecha del nombre.
-- Con LEFT JOIN incluimos todos los álbumes, incluso si no tuvieran canciones,
-- o si no registran ventas. Esto asegura que estén incluidos todos los álbumes.
-- Se agrupa (GROUP BY) por el nombre del artista, título del álbum y año de
-- lanzamiento para poder usar las funciones de agregación (COUNT y SUM) y
-- obtener un resumen para cada álbum.
```

```
-- Pregunta 17: Usar la vista recién creada para obtener el título, artista,
-- número de canciones e ingresos totales de los álbumes que hayan generado
-- más de 100 (dólares) en ingresos.

SELECT TituloAlbum, NombreArtista, NumeroCanciones, IngresosTotales
  FROM ResumenAlbumes
WHERE IngresosTotales > 100
 ORDER BY IngresosTotales DESC;

-- La vista se puede consultar como si fuera una tabla normal. Esto permite,
-- por ejemplo, reutilizar la consulta evitando reescribir la
-- lógica de los JOIN y GRUP BY.
```